Optimizing Code Creation:

A Comparative Analysis of AI-Generated Solutions

By: Benjamin Consalvo

Thesis Director: Matthew Sopha

Second Committee Member: Daniel Mazzola

Submitted to Barrett The Honors College

in fulfillment of the requirements for thesis certification

April 2024

Table of Contents

Introduction

The modern world is experiencing a surge in Artificial Intelligence (AI) influence. Technological advancements are happening daily, with AI applications permeating every aspect of life. College campuses are no longer unfamiliar sights for self-driving Waymo cars and Starship delivery robots. Social media is flooded with an abundance of AI-generated content, including recordings of celebrity speeches eerily crafted but never spoken. ChatGPT, for example, can write complex papers in a mere two minutes, a feat that would take the average student days to accomplish. These advancements beg the question: is there anything AI can't achieve? Can it truly surpass human capabilities in all tasks, even those requiring specialized training?

One such domain where human expertise remains crucial is coding. Often likened to learning a new language due to its complexity, coding forms the bedrock of websites, apps, and the very technology that drives AI itself. Building programs from scratch is a laborious process, even for trained professionals in Computer Information Systems (CIS). It can involve hours of trial and error. This begs a fascinating question: could AI coding generator software potentially create the same high-quality applications as these professionals, and do so in a fraction of the time?

Generative artificial intelligence represents a transformative technology that possesses the remarkable ability to create diverse forms of content, including text, images, audio, and video, in response to user prompts (U.S. GAO). Such generative AI systems, exemplified by prominent applications like ChatGPT and Gemini, hold immense promise for revolutionizing numerous sectors spanning education, governance, legal practice, and entertainment. As of early 2023, the widespread adoption of these emerging generative AI systems had already surpassed the milestone of 100 million users, with advanced chatbots, virtual assistants, and language translation tools serving as prime examples of their pervasive utility and global recognition for their beneficial applications (U.S. GAO).

Despite the undeniable advantages that generative AI brings, there are lingering concerns regarding its potential misuse. Chief among these concerns is the apprehension surrounding the replication of creators' works, which raises questions about intellectual property rights and artistic integrity. Moreover, the prospect of generative AI being harnessed to generate code for more sophisticated cyberattacks poses significant security challenges in an increasingly digitized world (U.S. GAO). Furthermore, there are ethical and safety implications associated with the potential use of generative AI in the production of new chemical warfare compounds and other malicious applications, highlighting the imperative for responsible development and deployment of such technology (U.S. GAO).

Generative AI's versatility extends across a vast array of disciplines, promising transformative applications in education, governance, medicine, law, and beyond. By leveraging user prompts, these systems can rapidly generate and refine outputs, ranging from crafting speeches in specific tones to distilling complex research findings and assisting in legal document analysis. Furthermore, generative AI's creative prowess extends to the realm of art, where it can produce visually stunning images for video games, compose musical pieces, and generate evocative poetic language—all from simple text inputs (U.S. GAO). Additionally, generative AI holds immense potential in facilitating complex design processes, such as molecular modeling for drug discovery and the automated generation of programming code, thus streamlining innovation in various industries.

At the heart of generative AI's capabilities lies its ability to learn intricate patterns and relationships from vast datasets, empowering it to generate content that closely resembles the underlying training data. These systems harness sophisticated machine learning algorithms, statistical methods that computers use to learn from data without being explicitly programmed, and statistical models to process and create content across multiple modalities (U.S. GAO). For instance, large-scale language models are trained on extensive corpora of text data to discern patterns in written language and emulate human writing styles convincingly. Furthermore, generative AI systems exhibit adaptability to diverse data types,

encompassing programming codes, molecular structures, and images, thereby broadening their applicability across a spectrum of domains (U.S. GAO).

The evolution of generative AI has been propelled by advancements in computing power, which have enabled the development of mature systems such as advanced chatbots, virtual assistants, and language translation tools that are now integral to various facets of daily life. As these technologies continue to mature, emerging generative AI systems are poised to unlock novel applications and address pressing societal challenges. For instance, research institutions are exploring the deployment of generative AI programs to automate responses to patient inquiries, thereby alleviating the administrative burden on healthcare providers and enhancing the overall patient experience (U.S. GAO). Similarly, companies are leveraging pre-trained generative AI models to enhance customer communication and personalize interactions in innovative ways.

Generative AI stands as a testament to the transformative potential of artificial intelligence, offering boundless opportunities for innovation and societal advancement. While the technology holds immense promise across diverse domains, it is imperative to address ethical, legal, and security considerations to ensure responsible development and deployment. By fostering collaboration among stakeholders and embracing a holistic approach to AI governance, we can harness the full potential of generative AI to create a more equitable, efficient, and sustainable future for humanity.

While there are many big world benefits and concerns regarding Generative AI, students in universities and lower education are able to utilize these AI's to help comprehend and complete homework in a deeper understanding. Specifically for me, the earlier days of college, Generative AI was not mainstream and rarely talked about until late 2022 when ChatGPT was launched. After ChatGPT was released, many syllabuses had to be revised and include a section on the ethical use of generative AI. Most classes were officially against the use of these AI's and found them to be plagiarism, while others like coding classes encouraged students to utilized them. Personally, it was more common for my teachers

to encourage them in the latter year of my college experience perhaps due to the realization that these AI chatbots are not going anywhere anytime soon and awareness that we must adapt to the technology of today. Knowing this, I wanted to put a few of these generative AI chatbots to the test to see whether they could create a complex Java program meeting intricate coding criteria with minimal errors. A comparative analysis of different Generative AI systems (ChatGPT, Gemini, BlackBox, Copilot) reveals variations in their capacity to generate such a program, highlighting the limitations of these tools for handling intricate coding criteria compared to human programmers.

Traditional Coding: An Application Case Study

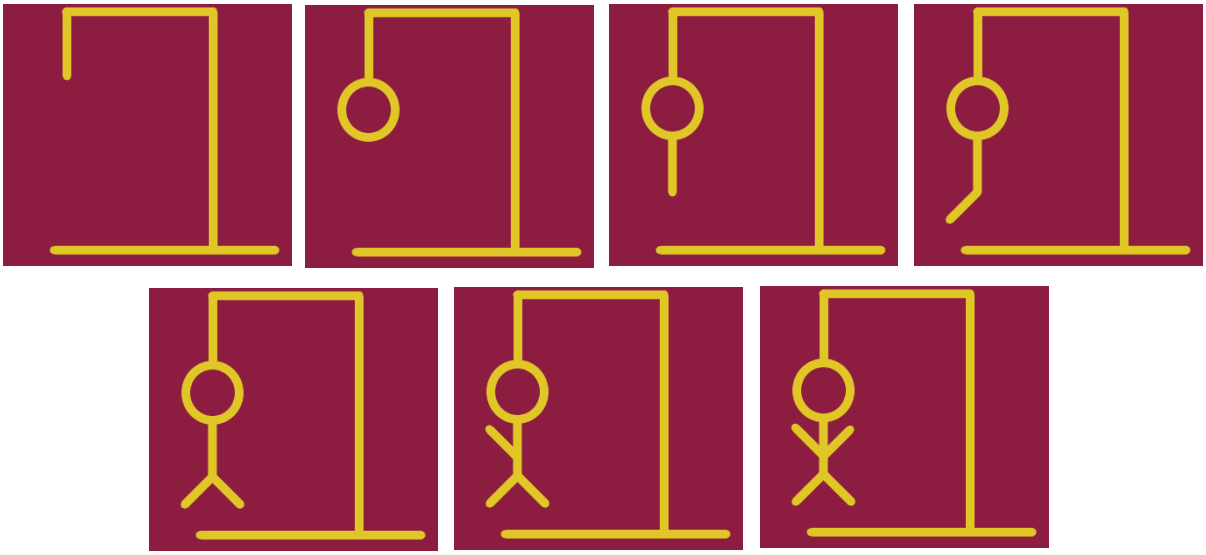
The success of this program can be attributed to the well-established waterfall approach used in its development. Unlike some more agile methodologies, waterfall emphasizes a structured, linear progression through distinct phases. This provided a clear roadmap for the entire project, minimizing the need for backtracking and revisions later on. For this, imagine a cascading waterfall, where each level feeds into the next.

The first stage of the waterfall approach is akin to the source of the waterfall - Requirements Gathering. Here, meticulous planning takes place. Every detail – functionalities, user needs, system constraints – is documented and solidified before moving forward. This initial investment in planning ensures a strong foundation for the subsequent phases. For my program, I made specific requirements that needed to be completed in order to have my program functioning properly. These requirements included:

- A fully functioning GUI by way of Java Swing.
- A way to press buttons that in turn check to see if the letter is in the word to guess.
- A section where once the button is pressed the letter is displayed to show which letters have been used previously.
- An area where you can visually calculate how many wrong guesses you have. I had in mind to use a series of pictures that build on the previous one.

The next stage translates those requirements into a detailed blueprint. Much like the powerful flow of water carving a path, the Design phase takes the gathered information and translates it into a concrete plan for the program's architecture and functionality. This blueprint serves as a guide for the next step. The main logic behind my program was in creating the random word, displaying the word and the underscores according to how long it is, and the button logic for each press. On top of this, I used Paint 3D to create a series of different hangman pictures that would be used sequentially throughout the

program. Shown below, are the pictures of the hangman and the pseudocode that helped me articulate my thoughts into a clear blueprint that was used in the coding process.



Pseudocode for underscores and hidden letters:

Create a random word and parse each letter into an array

For each letter in the random word created

---------Create a JLabel and label it with an underscore

---------Set the bounds and change the x-axis incrementally

---------Create a JLabel and label it with the letter of the word

---------Set the color of the label to be the same as the background

---------Set the bounds and change the x-axis incrementally to be equal to the underscores

---------Set the y-axis a little higher than the underscores

Pseudocode for button logic:

For each letter in an array that has the alphabet

---------Create a JButton and label it accordingly

---------Set the bounds and change the x-axis incrementally

---------Add an ActionListener for once the button is clicked

----------Set enabled too false

----------If letter pressed is in the word to guess

----------Change the color of the letter so it is visible above the correlated underscore

----------Else

----------Increase a wrong guess count by one

----------Switch (wrong guess count)

----------For each case numbering from 1-6:

----------Change the picture of the man hanging to accurately depict where the player is in hanging the man.

With a clear understanding of the requirements and a detailed design in hand, the development phase brings the program to life. This stage, akin to the cascading water itself, involves writing the actual code that makes the program function. Programmers, armed with the thorough plan from the previous phases, translate the design into functional code. For me, I had previous practice creating GUIs using Java Swing so having the knowledge along with the pseudocode for the logic of the program, I was able to create a rough draft of what I wanted it to look like. I will go more into detail of the code later in this section.

Once the code is written, rigorous testing ensures the program functions as intended. This testing phase resembles the churning water at the base of the waterfall, rigorously checking for cracks or weaknesses. Just as the churning water exposes any underlying issues in the rock face, testing exposes any bugs or errors in the code. For me, there was a decent amount of trial and error for placing all of the Java Swing components and testing where I wanted them all to fit together. All things considered, the program itself was not overly complicated outside of the placement of the Java Swing components so I had little to no errors and bugs to work through.

Finally, the program is deployed and delivered to the users, much like the water reaching its destination. However, the waterfall approach doesn't end there. Maintenance is an ongoing process, akin

to the continuous flow of the waterfall. This ensures the program remains functional, addressing bugs, implementing new features, and adapting to changing needs over time. This final step for me was the deployment of my program into this paper and the ability to use it to compare versus the AI's generated code.

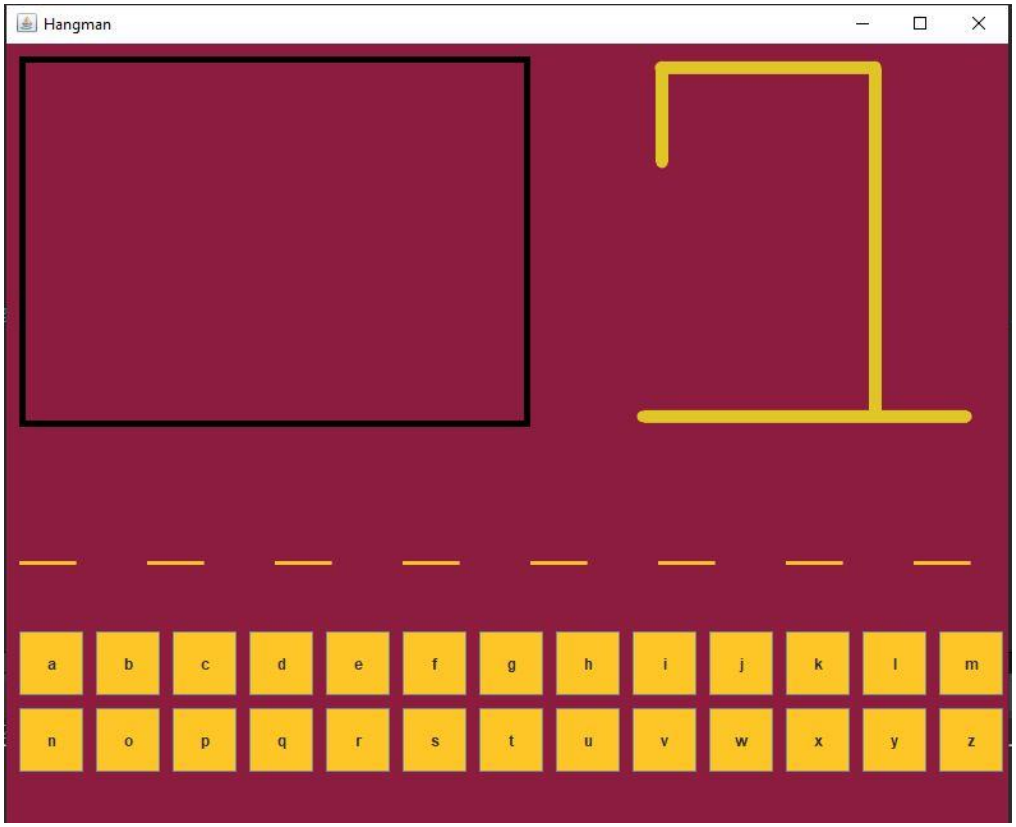


Figure 1

To start, looking at Figure 1, this is the completed application (GUI) prior to being played. It includes a strike-out box in the top left corner, a home for all letters once they have been guessed. The gallows in the top right corner for the potential hangman victim. Guess lines for the correct letters to be placed in the center of the GUI, and a button for each letter in the alphabet.

```
14 public class Hangman extends JFrame implements ActionListener {
15
16     int xb = 10;
17     int yb = 10;
18     int xletters = 10;
19     int yletters = 10;
20     int newLine = 0;
21     int xlines = 10;
22     int ylines = 10;
23     int ylineletters = 5;
24
25     int wrongGuesses = 0;
26     int rightGuesses = 0;
27
28     Color maroon = (new Color(140,29,64));
29     Color gold = (new Color(255,198,39));
30
31     String[] words = {"banana", "apple", "pear", "orange", "grape", "observer", "grumpy", "savvy", "prepare", "disgust", "enormous", "mind", "sight",
32                       "spring", "tiger", "insert", "rustic", "cemetery", "salt", "soap", "impolite", "birthday", "brother", "sweater", "migrate"};
33     Random random = new Random();
34
35     int randomNumber = random.nextInt(words.length);
36     String word = words[randomNumber];
37     int length = words[randomNumber].length();
38     String[] arrayOfWord = new String[word.length()];
39     JLabel[] individualLetters = new JLabel[length];
40
41
42     ImageIcon step1 = new ImageIcon("step1.png");
43     ImageIcon step2 = new ImageIcon("step2.png");
44     ImageIcon step3 = new ImageIcon("step3.png");
45     ImageIcon step4 = new ImageIcon("step4.png");
46     ImageIcon step5 = new ImageIcon("step5.png");
47     ImageIcon step6 = new ImageIcon("step6.png");
48     ImageIcon step7 = new ImageIcon("step7.png");
49     JLabel picture = new JLabel(step1);
```

Figure 3

Figure 2 shows the global variable bank necessary for all cross referencing throughout the program. Notably, lines 31 through 39 establishes the randomization of the correct word associated with each round of the game utilizing the java.util.Random library.

*Main.java × Hangman.java

```
1
2  public class Main {
3
4      public static void main(String[] args) {
5
6          new Hangman();
7
8      }
9
10 }
```

Figure 2

Looking at Figure 3, This is the Main Class, separate from the code, where the main method is located, creating and calling the instance of the Hangman class.

```
51    Hangman() {
52
53        //Frame
54        this.setSize(800,650);
55        this.setTitle("Hangman");
56        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
57        this.setResizable(true);
58        this.setLocationRelativeTo(null);
59        this.getContentPane().setBackground(maroon);
60        this.setVisible(true);
61        this.setLayout(null);
62
63        //Letter display box
64        JPanel displayArea = new JPanel();
65        displayArea.setBounds(10, 10, 400, 290);
66        displayArea.setBackground(maroon);
67        displayArea.setBorder(BorderFactory.createLineBorder(Color.BLACK, 5));
68        displayArea.setLayout(null);
69
70        //Place where the "__" are
71        JPanel wordPanel = new JPanel();
72        wordPanel.setBounds(0, 350, 800, 100);
73        wordPanel.setBackground(maroon);
74        wordPanel.setLayout(null);
75
76        //Hang-man pictures
77        JPanel hangman = new JPanel();
78        hangman.setBounds(440, 10, 330, 300);
79        hangman.setLayout(null);
80
81        picture.setBounds(0, 0, 330 , 300);
82        hangman.add(picture);
83
84        //Button Area
85        JPanel buttonsPanel = new JPanel();
86        buttonsPanel.setBackground(maroon);
87        buttonsPanel.setBounds(0, 450, 800, 130);
88        buttonsPanel.setLayout(null);
```

Figure 4

Shown in Figure 14 and utilizing Java Swing, the Frame and Panels are created first after creating the constructor. To start, the Frame includes the deciders of location of the application's details, their sizing, their visibility, and the background color. Second, was the Letter Display Box, which created the strike-out box referenced in Figure . Next is the JPanel where the underscores are placed for the correct letter choices throughout the gameplay. Fourth, is the Hangman JPanel that calls the global JLabel variable "picture" (see line 49, Figure 2) and adds it to the Hangman JPanel. Finally, I crafted the JPanel where the alphabet buttons were placed.

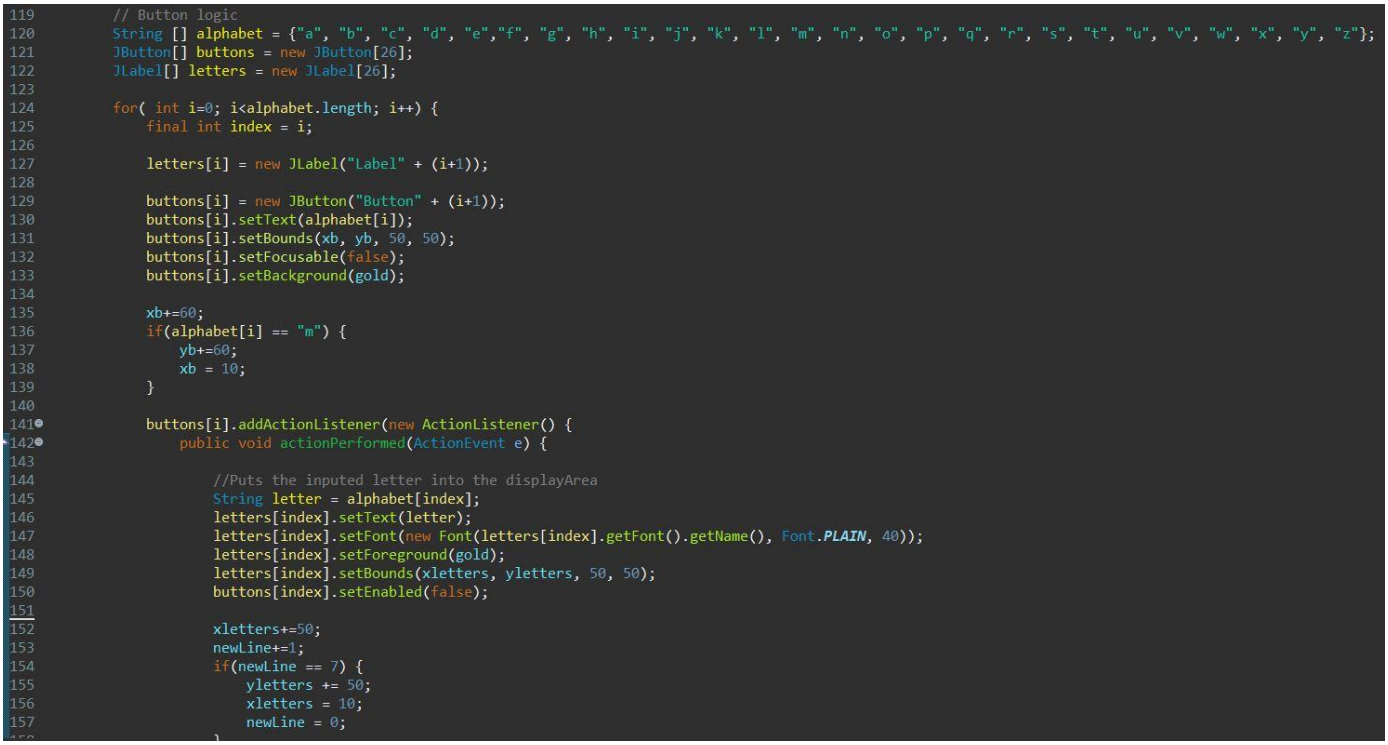
```
91          // Parsing the random word
92          for (int i = 0; i < word.length(); i++) {
93              arrayOfWord[i] = String.valueOf(word.charAt(i));
94          }
95
96          // Creates the lines and the hidden word to guess
97          for(int i=0; i<length;i++) {
98
99              //lines
100             JLabel lines = new JLabel();
101             lines.setText("__");
102             lines.setForeground(gold);
103             lines.setFont(new Font(lines.getFont().getName(), Font.PLAIN, 40));
104             lines.setBounds(xlines, ylines, 100, 50);
105
106             //Letters
107             individualLetters[i] = new JLabel("Label" + (i+1));
108             individualLetters[i].setText(arrayOfWord[i]);
109             individualLetters[i].setForeground(maroon);
110             individualLetters[i].setFont(new Font(lines.getFont().getName(), Font.PLAIN, 40));
111             individualLetters[i].setBounds(xlines + 10, ylineletters, 100, 50);
112
113             xlines+=100;
114
115             wordPanel.add(individualLetters[i]);
116             wordPanel.add(lines);
117         }
```

Figure 5

On line 92 of Figure 5, a for loop was created to parse the random word chosen (see line 38, Figure 2) and separated each letter of said word. Next, another for loop was created that will repeat the process with each letter of the word chosen; firstly constructing a single underscore that corresponds to the letter linked to the index value “i” of the individualLetters array. Then I create a series of JLabels each correlating to the individual letters and placing them in the same area as each underscore. Notably, each letter starts as maroon, like the background, to hide it until is unvailed by pressing the correct button, changing it to gold. Finally, I increased the x-axis for both the lines and the letter by 100 pixels in order to create aesthetic spacing in the GUI.

Figures 6-8

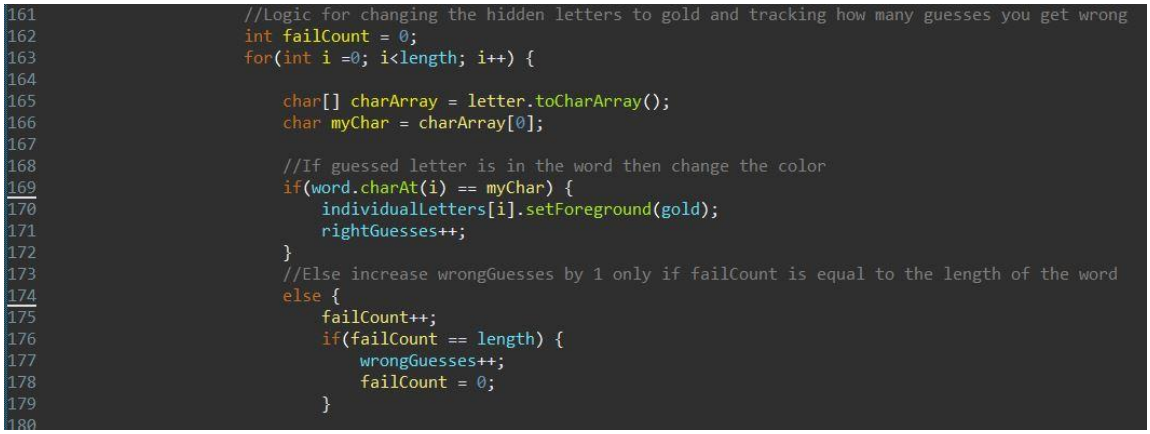
These Figures are all encased in the for loop created on line 124 for the purpose of the logic behind each button in the program.



```
119         // Button logic
120         String [] alphabet = {"a", "b", "c", "d", "e","f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z"};
121         JButton[] buttons = new JButton[26];
122         JLabel[] letters = new JLabel[26];
123
124         for( int i=0; i<alphabet.length; i++) {
125             final int index = i;
126
127             letters[i] = new JLabel("Label" + (i+1));
128
129             buttons[i] = new JButton("Button" + (i+1));
130             buttons[i].setText(alphabet[i]);
131             buttons[i].setBounds(xb, yb, 50, 50);
132             buttons[i].setFocusable(false);
133             buttons[i].setBackground(gold);
134
135             xb+=60;
136             if(alphabet[i] == "m") {
137                 yb+=60;
138                 xb = 10;
139             }
140
141             buttons[i].addActionListener(new ActionListener() {
142                 public void actionPerformed(ActionEvent e) {
143
144                     //Puts the inputed letter into the displayArea
145                     String letter = alphabet[index];
146                     letters[index].setText(letter);
147                     letters[index].setFont(new Font(letters[index].getFont().getName(), Font.PLAIN, 40));
148                     letters[index].setForeground(gold);
149                     letters[index].setBounds(xletters, yletters, 50, 50);
150                     buttons[index].setEnabled(false);
151
152                     xletters+=50;
153                     newLine+=1;
154                     if(newLine == 7) {
155                         yletters += 50;
156                         xletters = 10;
157                         newLine = 0;
```

Figure 6

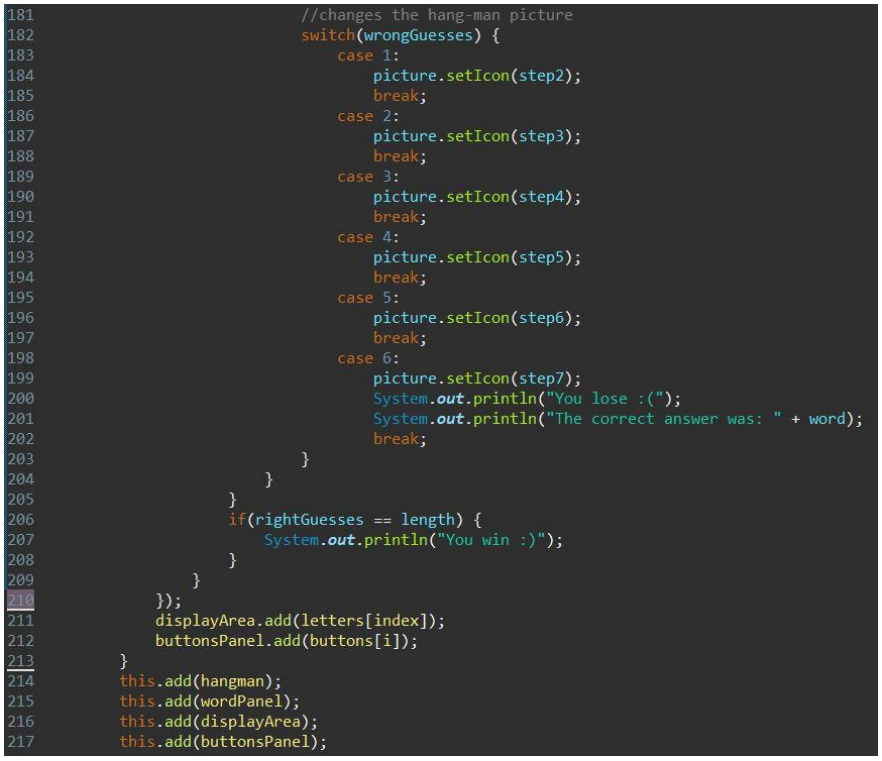
The first section of the for loop shown in Figure 6 (lines 127-139), creates and places each button, setting the text to the correct index value of "i" inside the alphabet String Array on line 120. Specifically, on line 135-139, each time a button is placed the x-axis is increased by 60 pixels. In addition to this, if the index value of "alphabet" is equal to "m" it also adjusts the y-axis by 60 pixels to allow for the movement to the next line of letters. After moving 60 pixels on the y-axis, it resets the x-axis to the original value of 10 pixels. On lines 141 and 142, the event handler ActionListener is created and attached to each button. Inside of this ActionListener, on lines 145-158, I firstly composed the logic that each time the button is pressed it displays the specific letter inside the strike-out box in the top left corner.



```
161                 //Logic for changing the hidden letters to gold and tracking how many guesses you get wrong
162                 int failCount = 0;
163                 for(int i =0; i<length; i++) {
164
165                     char[] charArray = letter.toCharArray();
166                     char myChar = charArray[0];
167
168                     //If guessed letter is in the word then change the color
169                     if(word.charAt(i) == myChar) {
170                         individualLetters[i].setForeground(gold);
171                         rightGuesses++;
172                     }
173                     //Else increase wrongGuesses by 1 only if failCount is equal to the length of the word
174                     else {
175                         failCount++;
176                         if(failCount == length) {
177                             wrongGuesses++;
178                             failCount = 0;
179                         }
180
```

Figure 7

Next on line 163 of Figure 7, another for loop is created which keeps track of the number of incorrect guesses collected throughout the gameplay, correlating to the creation of the hangman on the gallows. This is known as the variable failCount on line 162. If this variable is equal to the length of the randomized word, the variable names wrongGuesses is increased by one. In addition, if the correct letter is guessed, the corresponding letter is changed to gold on its wordPanel and increases the variable rightGuesses by one.



```
181                         //changes the hang-man picture
182                         switch(wrongGuesses) {
183                             case 1:
184                                 picture.setIcon(step2);
185                                 break;
186                             case 2:
187                                 picture.setIcon(step3);
188                                 break;
189                             case 3:
190                                 picture.setIcon(step4);
191                                 break;
192                             case 4:
193                                 picture.setIcon(step5);
194                                 break;
195                             case 5:
196                                 picture.setIcon(step6);
197                                 break;
198                             case 6:
199                                 picture.setIcon(step7);
200                                 System.out.println("You lose :(");
201                                 System.out.println("The correct answer was: " + word);
202                                 break;
203                         }
204                     }
205                 }
206                 if(rightGuesses == length) {
207                     System.out.println("You win :)");
208                 }
209             }
210         });
211         displayArea.add(letters[index]);
212         buttonsPanel.add(buttons[i]);
213     }
214     this.add(hangman);
215     this.add(wordPanel);
216     this.add(displayArea);
217     this.add(buttonsPanel);
```

Figure 8

In this last section of the code shown in Figure 8, a switch is created to correlate the number of wrong guesses with the alternation of the gallows photos. For every wrong guess, more of the hangman

victim is revealed by a new picture set within the case of the switch. If wrongGuesses equals six, then the game ends and the console will display a “you lose :(” message, providing the player with the correct answer. Finally, if the rightGuesses variable is equal to the length of the word, the game ends and the console will display “you win :)”.

AI Versus Human Expertise: Prompt

As shown in the figures above, this project was a great workload that without my prior knowledge in the field, might have taken someone days to complete successfully. With the recent increase of everyday AI usage, I decided it was time to conclude which Ais are better at coding than others. The AIs that were chosen to create this program are the following:

- Gemini
- Copilot
- BlackBox
- ChatGPT

First things first there are a few constants that span across all AI:

1. To give each AI the same chance of success, I gave each the same prompt of:

   > "Utilizing Java Swing, create the game hangman with a background color of Color(140, 29, 64), buttons colored Color(255, 198, 39) for each letter of the alphabet at the bottom that can only be clicked once, an area to display the incorrect letters once they have been clicked in the top left, an area in the middle to display "_", with a space between each "_", for each of the correct letters of the random word, and an area in the top right where it switches between 7 png (named hangman0-6) pictures, located in the same project file, based off of how many incorrect guesses ending the game if it reaches 6 wrong guesses"

2. I gave each AI the same hangman pngs that I used in my own code.

AI-Generated Code Solution Comparison: Failures

Gemini

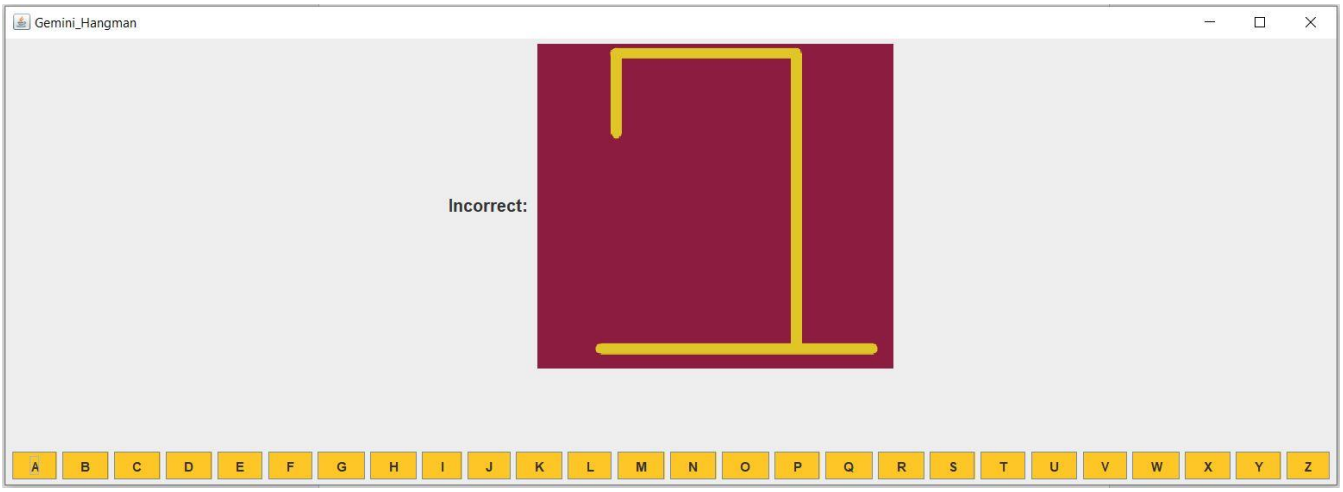


Figure 9

The first AI put on trial was Google's Gemini. Something that Gemini did that is good practice as a software developer is the use of methods. Not including the main method, it made a total of 8 methods that were called throughout the program. This not only improves the readability of the code but also creates less redundant sections of code if the program grew into something bigger. This utilization of methods highlights the ability on Gemini's part to understand and create efficient code. On top of this, as shown in Figure 9 above, Gemini successfully created a working GUI using Java Swing but had some unseen challenges that resulted in the program to ultimately not work properly. Additionally worth noting but not crucial to the code, when asking Gemini the prompt, it failed to give the whole code in one group. I had to ask it to give me the rest of the code in a different prompt creating two separate groups of code.

```
19    public Gemini_Hangman() {
20        super("Gemini_Hangman");
21        setSize(1250,450);
22        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23        setLayout(new BorderLayout());
24
25        // Set background color
26        getContentPane().setBackground(new Color(140, 29, 64));
27
28        // Create panels for different sections
29        JPanel topPanel = new JPanel();
30        JPanel centerPanel = new JPanel();
31        JPanel bottomPanel = new JPanel();
```

Figure 10

In Figure 10, Gemini decided to omit the setSize() method of the JFrame on line 21. I had to manually input a setSize() to display the frame properly. This small but important failure on Gemini's part is surprising due to how simple the request was. The fact that it was unable to create one of the most important line highlights it inadequacy in the finer details of coding. On top of this, even though it was technically successful in setting the background to a specific color, Gemini decided to create three different JPanels for each section of the GUI, located on lines 29-31. Throughout the code, the JLabels and JButtons created were placed onto each of these JPanels instead of placing them directly onto the JFrame. This in turn completely covered the base background of the GUI creating the illusion that the background color was not set. This failure showcases the idea that Generative AI will do what you ask but can override what you specifically have in mind if it thinks of a different way to go about the structure.

```
84    private void startGame() {
85        currentWord = words[new Random().nextInt(words.length)];
86        guessedLetters = new ArrayList<>();
87        incorrectGuessesCount = 0;
88        updateWordDisplay('_');
89        incorrectGuesses.setText("Incorrect: ");
90        updateHangmanImage();
91        enableLetterButtons();
92    }
93
94    private void updateWordDisplay(char guess) {
95        StringBuilder hiddenWord = new StringBuilder(wordDisplay.getText());
96        for (int i = 0; i < currentWord.length(); i++) {
97            if (currentWord.charAt(i) == guess) {
98                hiddenWord.setCharAt(i, guess);
99            }
100       }
101       wordDisplay.setText(hiddenWord.toString());
102   }
```

Figure 11

Another thing that failed was trying to display an underscore for each letter in the word to guess. As shown in Figure 11, in the method startGame(),which is called on loading the program, on line 88 it calls the method updateWordDisplay(). This passes '_' as a parameter which logically does not make

sense since the parameter is supposed to be the guessed letter, not an underscore. This failure highlights the logically imperfections of an AI trying to create a systematic and functional method that reads user inputs, in this case the pressing of a button.

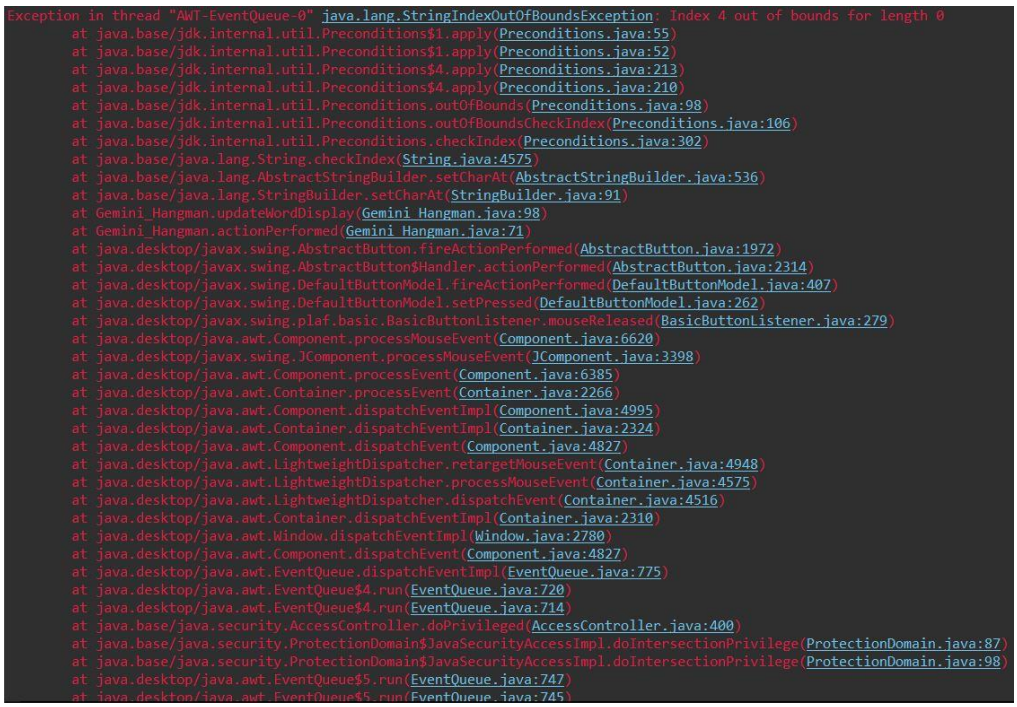


Figure 12

The biggest failure happened when a correct button was pressed. This huge error shown in Figure 12 above is displayed in the console giving a StringIndexOutOfBoundsException. This error, similar to the underscore display problem, happened inside the updateWordDisplay() method. On line 98 of Figure 11, when the correct button is pressed, it looks into the hiddenWord String and tries to find the guessed letter. The index error occurs due to the fact that the hiddenWord String is failed to be set to anything. On top of this, on the actual GUI of the game, nothing updates or changes. This error makes the game unplayable showcasing the inadequacies of Gemini's ability to make reliable, complex, and logical methods of code.

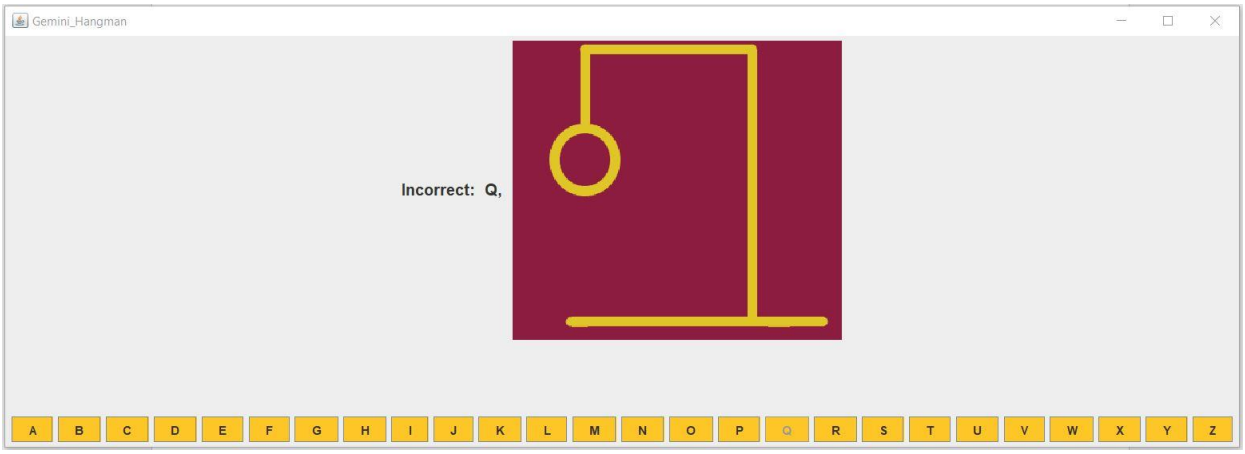


Figure 13

Although there were a lot of failures in this code, there were a few things that succeeded. Seen in Figure 13, the three main successes were that Gemini made it so the buttons could not be pressed more than once (letter "Q"), the incorrect letters were displayed in a group, and the image accurately updated for every wrong guess. I found it interesting that it could easily add the line of code to setEnabled(false) for each button but failed to setSize() of the frame. Even with these aspects functioning, there were not enough working pieces of this program to call it a success, mainly due to the index error made it unplayable with every correct guess of the letter.

Copilot

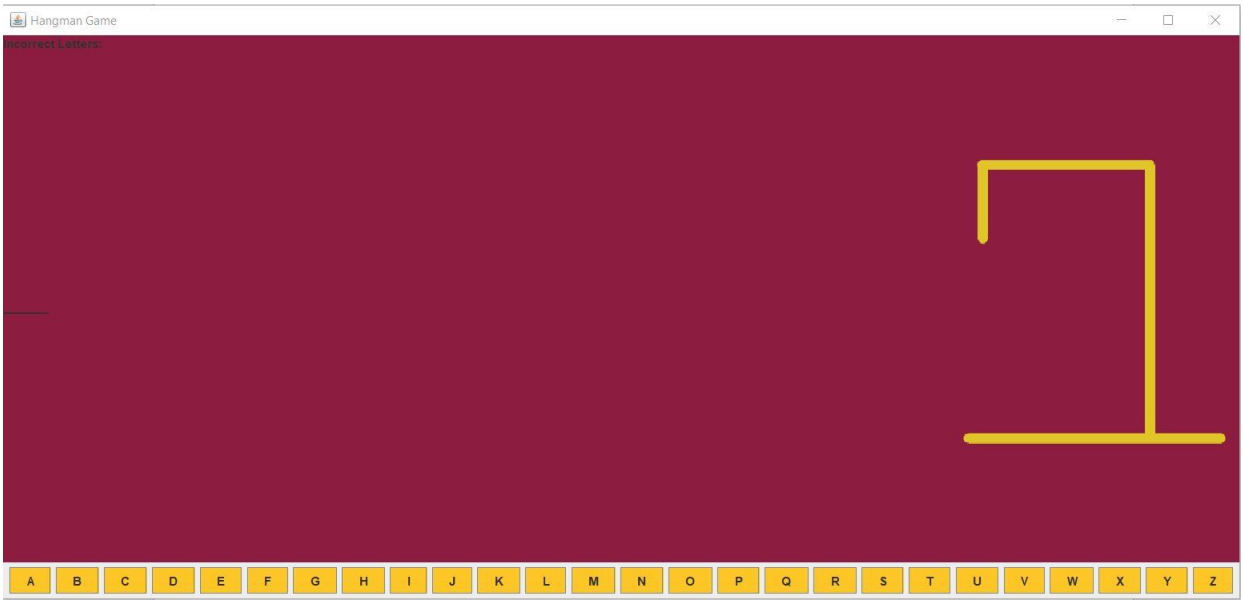


Figure 14

Looking at Figure 14, Copilot did a few things better than Gemini. For starters, the background color was successfully displayed and, although very small and hard to see, the underscores displayed albeit without spaces between them. On top of this, the incorrect letters section in the top left was correctly displayed but has a critical error that we will get into later. One thing I had to do similarly to Gemini was adjust the setSize() to properly display the full GUI. Different from Gemini though, it displayed the GUI, but the GUI before adjustment did not show all of the buttons due to them being set in one long row.

```
26          // Initialize game state
27          currentWord = getRandomWord();
28          guessedWord = new StringBuilder("_".repeat(currentWord.length()));
29          incorrectGuesses = 0;
```

Figure 15

I found the most interesting part of the code to be the way that copilot rendered the underscores. Shown in Figure 15, Copilot utilized the method StringBuilder() to get the length of the random word and replace each character with an underscore. This seemed to be the most efficient way the underscores were created compared to all the other AIs, highlighting Copilot's ability to create efficient and working code. This is different from how I went about creating the underscores.

```
33          for (char c = 'A'; c <= 'Z'; c++) {
34              JButton button = new JButton(String.valueOf(c));
35              button.setBackground(new Color(255, 198, 39));
36              button.addActionListener(new ActionListener() {
37                  @Override
38                  public void actionPerformed(ActionEvent e) {
39                      handleLetterClick(button.getText());
40                  }
41              });
42              buttonPanel.add(button);
43          }
```

Figure 16

Looking at Figure 16, another aspect that I found quite interesting was the similar but different implementation in regard to my code of the ActionListener() inside of the for loop that created the buttons. Differently, Gemini and later talked about ChatGPT separated the actionPerformed() method from the buttons' for loop. The most unique part of this section of code was that Copilot used a combination of both my way, where I wrote all the code inside the actual actionPerformed() method, and the ways that the other AIs went about this section. You can see that inside the ActionPerformed() there was only one line of code which was a method called handleLetterClick(). I found this to be more efficient than my own code simply due to the fact that Copilot utilized method building and calling. Looking back on my own code, it is clear that if this grew into something bigger and more complex, my lack of method building would hurt me in the long run. This in turn is where Copilot got it better and succeeded over the human.

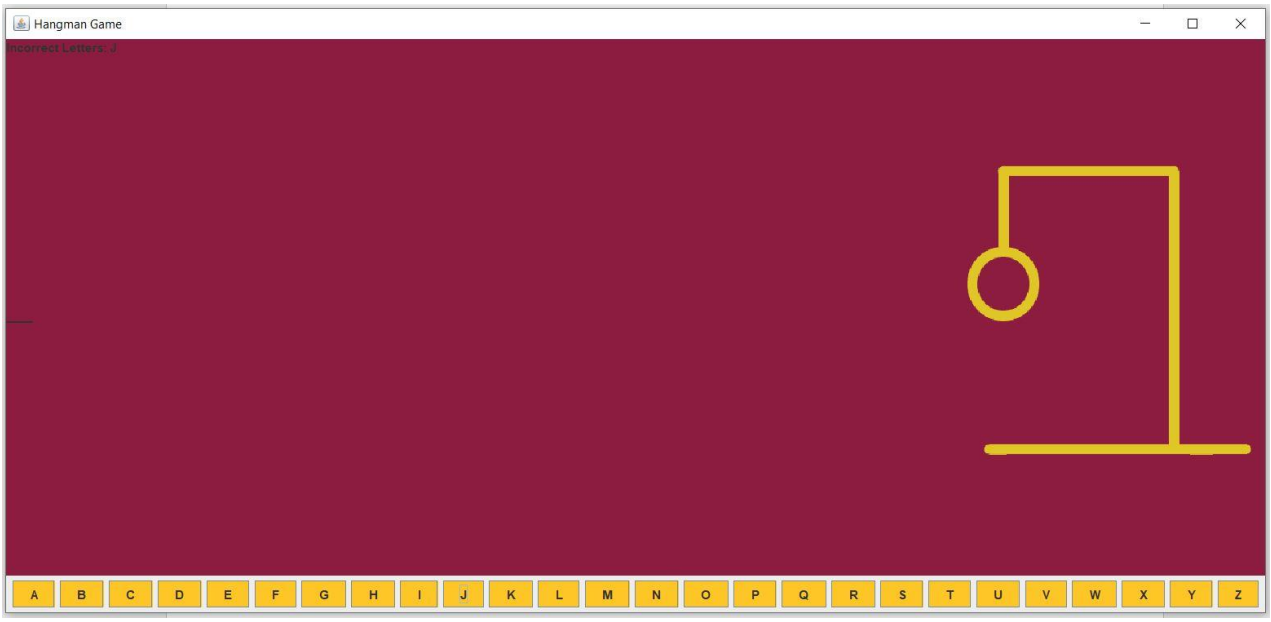


Figure 17

Although there were some interesting and unique things about the code, the functionality lacked in a lot of places. For the game in figure 17 I had already pressed the letter "J" and the word to guess was "java". Although hard to see, in the Incorrect Letters area in the top left, it outputted the guessed letter "J". In addition to this, if it was programed correctly, there should not be a head on the guy, the "J" should appear where the underscores are, and the "J" should not be displayed on the top left. Differently from my own code, there was no coded setEnabled(false) to stop me from pressing a button that was already pressed again. I was able to press the "J" button again if I wanted to which was incorrectly programed to have created another body part of the guy hanging.

```
84          } else {
85              // Incorrect guess
86              incorrectGuesses++;
87              incorrectLabel.setText("Incorrect Letters: " + letter);
88              hangmanImageLabel.setIcon(new ImageIcon("hangman" + incorrectGuesses + ".png"));
89              if (incorrectGuesses >= MAX_INCORRECT_GUESSES) {
90                  // Player loses
91                  JOptionPane.showMessageDialog(frame, "Game over! The word was: " + currentWord);
92                  System.exit(0);
93              }
94          }
```

Figure 18

To go deeper into the faulty section in the top left of the GUI titled Incorrect Letters, looking at Figure 18 and on line 87, the use of setText() caused the area to constantly be updating with the letter pressed instead of creating a catalog that displays all incorrect letters pressed. setText() is overriding the previously displayed letter and concatenating the new one with the string "Incorrect Letters: ".

BlackBox

```
217    private ImageIcon getLivesImage() {
218
219        String filename = "hangman" + (7 - lives) + ".png";
220
221        ImageIcon icon = new ImageIcon(getClass().getResource(filename));
222
223        return icon;
224
225    }
```

Figure 19

Before anything, there was a NullPointerException error that caused the GUI to not display on the screen. What I had to change in the code to make the GUI actually appear was to fix a pathing error for the images on line 221 of Figure 19. I replaced "getClass().getResource(filename));" inside the ImageIcon() to just "filename". I did this because all the images were already in the project folder with the .java and it was unnecessary to call the getClass() method. After this I was able to display the GUI, but I was quick to notice other errors.

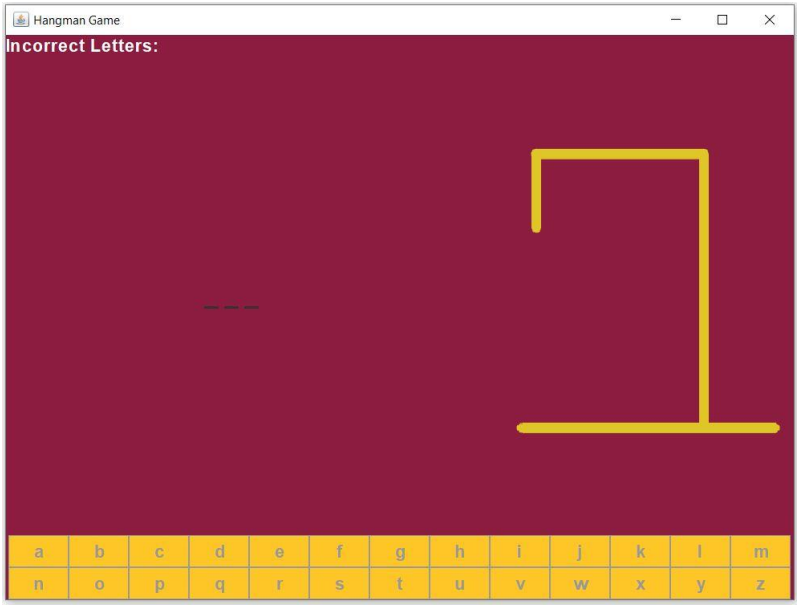


Figure 20

Looking at Figure 20, you can see the correctly displayed GUI. At first glance it looks quite promising with the correct background color, the underscores properly placed, and the image being properly displayed, but the moment I tried to click any of the buttons I realized that they were all disabled

by default. I find it interesting that BlackBox would automatically set all of the buttons to false when logically that does not make any sense. It is clear that BlackBox thrives on the visual side of the coding but logically is unable to keep up with the instructions or with a human. Once setting the buttons to enabled I was finally ready to try the game out.

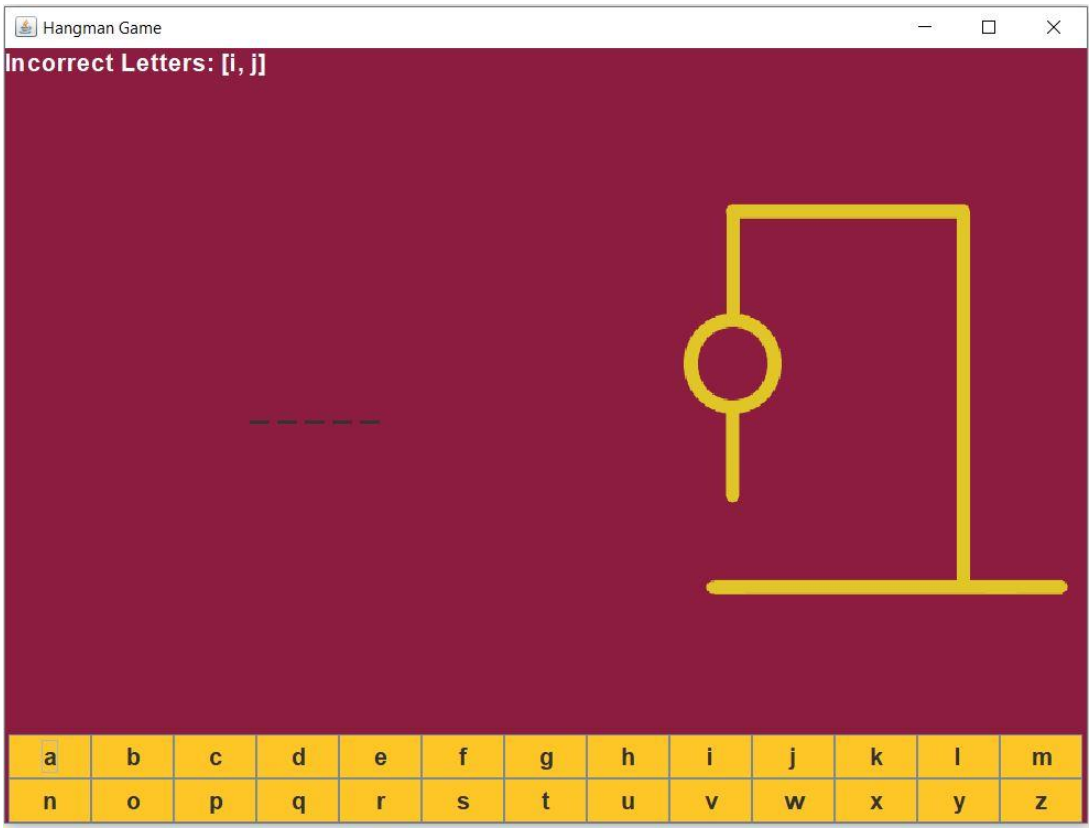


Figure 21

For the game in Figure 21, the random word chosen was “apple”. I started the game with the letters “I” and “J” and was pleasantly surprised it not only correctly displayed both in the top right, differently than Copilot, but also updated the image to accurately display the person hanging. The main problem of this application came up when I tried to click the “A”, and nothing happened.

```
186    private String getWordDisplay() {
187
188        StringBuilder sb = new StringBuilder();
189
190        for (char c : word.toCharArray()) {
191
192            if (incorrectLetters.contains(c)) {
193
194                sb.append(c);
195
196            } else {
197
198                sb.append("_ ");
199
200            }
201
202        }
203
204        return sb.toString();
205
206    }
```

Figure 22

Looking into the code on Figure 22, whenever you would press one of the buttons of the letter in the word to guess, the method getWordDisplay() would be called but it would never correctly append(c), which would replace an underscore, and would always go straight to the else statement. This happens due to the if statement on line 194 to update the underscores if the array incorrectLetters contains the letter that is pressed. Since the correct letter is not added to the incorrectLetters array, it cannot append(c). This in turn causes the game to be unwinnable and never allowing you to guess what the random word is. As stated previously, it is clear that BlackBox does a great job with the actual Java Swing GUI programing but fails with the functionality side of it all.

```
89          letterButtons = new JButton[26];
90
91          for (int i = 0; i< 26; i++) {
92
93              final int index = i;
94
95              char c = (char) ('a' + i);
96
97              letterButtons[i] = new JButton(String.valueOf(c));
```

Figure 23

Something I found interesting was that BlackBox was the only AI to create the buttons like I did. On line 89 of Figure 23, similar to me, BlackBox created an array[26] for each button and then used a normal for loop to increase the index each time.

AI-Generated Code Solution Comparison: Success

ChatGPT:

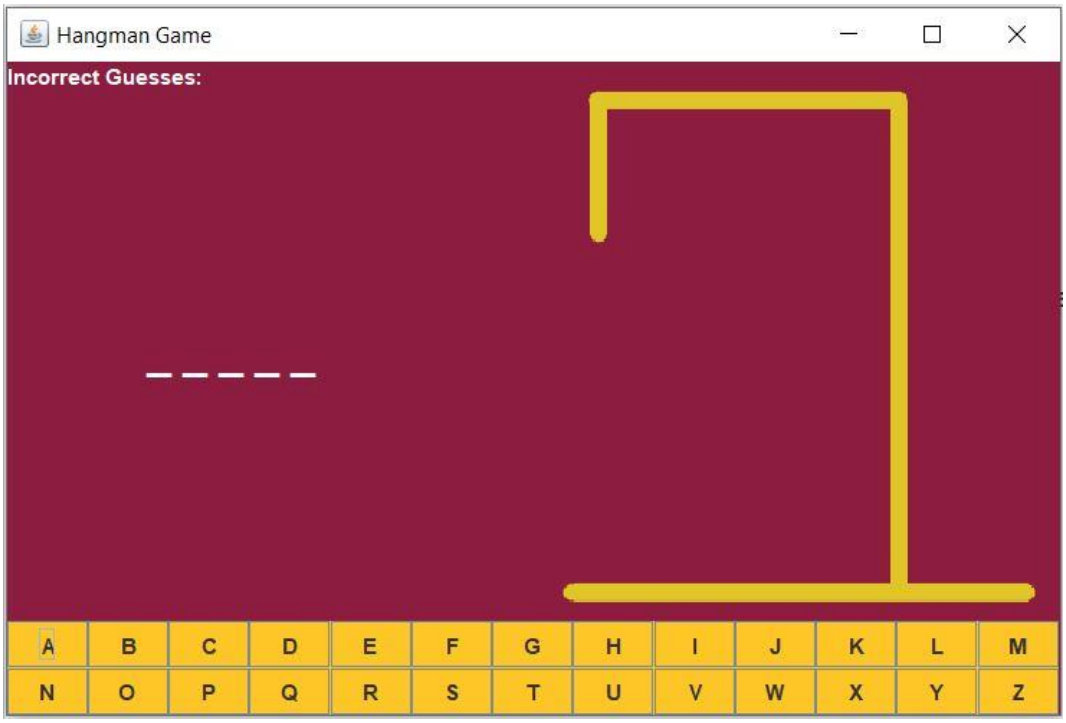


Figure 24

The most successful AI I found to produce adequate code with little to no errors was ChatGPT 3.5. Shown above in Figure 24, is the base GUI before any inputs via the buttons. Notably, similar to my code and BlackBox's, ChatGPT decided it was a good idea to split the buttons onto two rows after the letter "M". On top of this, a small but pleasant quality of life in this program compared to the others was the use of white text for the underscores, correct letters and incorrect letters. I did not specifically state to make those white so it is clear that ChatGPT was able to discern that the background and the letters would clash if left to the default black. This shows that ChatGPT has a high level of independent understanding and has the ability to think like a human in a way that the other AIs fail to.

```
38          JPanel buttonPanel = new JPanel(new GridLayout(2, 13));
39          buttonPanel.setBackground(new Color(140, 29, 64));
40          letterButtons = new ArrayList<>();
41          for (char c = 'A'; c <= 'Z'; c++) {
42              JButton button = new JButton(String.valueOf(c));
43              button.addActionListener(this);
44              button.setBackground(new Color(255, 198, 39));
45              buttonPanel.add(button);
46              letterButtons.add(button);
47          }
48          add(buttonPanel, BorderLayout.SOUTH);
```

Figure 25

Going a little deeper into the code, I found it very interesting that ChatGPT and Copilot used the same process to create the buttons and set a letter on each. Looking at Figure 25 above, you can see on line 41, they made use of a for loop that iterated through each letter of the alphabet and later added each on line 46 to the letterButtons Array list. As stated previously in Copilot's section, ChatGPT decided to have the ActionPerformed() method on the outside of the for loop and just call it inside of it. My method and Gemini's / ChatGPT's methods gave the same end result but utilized different techniques to achieve it. I would say that the AIs were more efficient and better practice due to them once again utilizing methods more than I did. Additionally, perhaps the most efficient line of code compared to mine I found was line 38, where ChatGPT ingenuously used a GridLayout() of (2,13). This is completely different from the way I did it previously stated when talking about Figure 6. I individually moved each button a certain number of pixels not realizing that GridLayout() existed. Essentially GridLayout() did all the work for ChatGPT and laid out each button an equal distance in the two rows. One more thing that is worth noting about this is that if you readjust the window, the buttons automatically change size and adjust accordingly. When adjusting my own program, the buttons do not follow with you and creates a big empty maroon space.

```
104     private void updateHangmanImage(int wrongGuesses) {
105         ImageIcon hangmanImage = new ImageIcon("hangman" + wrongGuesses + ".png");
106         hangmanLabel.setIcon(hangmanImage);
107     }
```

Figure 26

Another more efficient part of the code compared to mine was the way ChatGPT handled updating the image of the man hanging. Seen above on Figure 26, it dedicated a whole method to updating the hangman image and called it if the letter guessed was not in the chosen word. Looking back at my code I can definitely see that this is a much more compact and better use of memory to achieve the same thing as my switch case. Both mine and ChatGPT's technique in updating the image produced the same output but AI won in terms of efficiency.

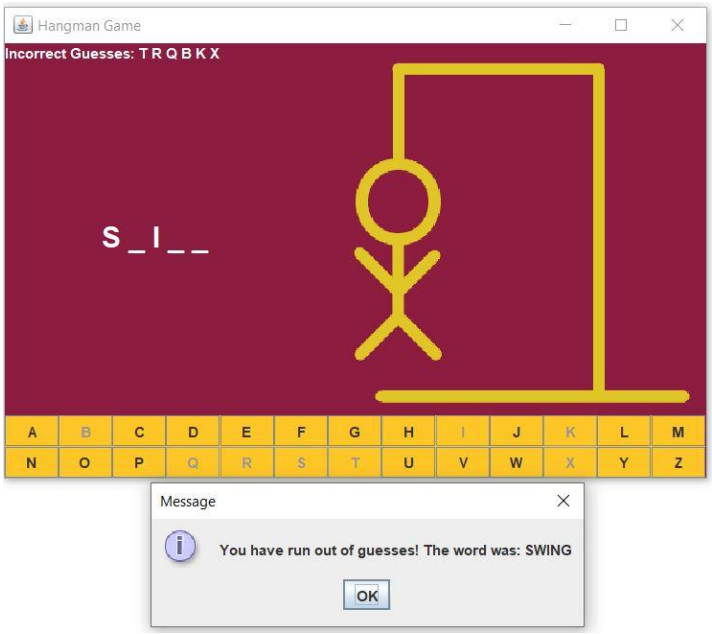


Figure 27

The only error I could find in ChatGPT's code had to do with the updating of the image. Although the efficiency in Figure 26 was the right way to go about updating the images, looking at Figure 27, once someone loses or wins and presses the "OK", the image did not reset to the starting hangman0.png. Only after playing the next game and getting a letter wrong did it update skipping hangman0.png and going directly to hangman1.png.

```
17    public ChatGPT_Hangman() {
18        setTitle("Hangman Game");
19        setSize(600, 400);
20        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21        setLayout(new BorderLayout());
22        getContentPane().setBackground(new Color(140, 29, 64));
23
24        hangmanLabel = new JLabel();
25        hangmanLabel.setHorizontalAlignment(SwingConstants.CENTER);
26        updateHangmanImage(0);
27        add(hangmanLabel, BorderLayout.EAST);
28
29        wordLabel = new JLabel("", SwingConstants.CENTER);
30        wordLabel.setFont(new Font("Arial", Font.BOLD, 24));
31        wordLabel.setForeground(Color.WHITE);
32        add(wordLabel, BorderLayout.CENTER);
33
34        incorrectLabel = new JLabel("Incorrect Guesses: ", SwingConstants.LEFT);
35        incorrectLabel.setForeground(Color.WHITE);
36        add(incorrectLabel, BorderLayout.NORTH);
37
38        JPanel buttonPanel = new JPanel(new GridLayout(2, 13));
39        buttonPanel.setBackground(new Color(140, 29, 64));
40        letterButtons = new ArrayList<>();
41        for (char c = 'A'; c <= 'Z'; c++) {
42            JButton button = new JButton(String.valueOf(c));
43            button.addActionListener(this);
44            button.setBackground(new Color(255, 198, 39));
45            buttonPanel.add(button);
46            letterButtons.add(button);
47        }
48        add(buttonPanel, BorderLayout.SOUTH);
49
50        startNewGame();
51    }
```

Figure 28

The error talked about above happens because once the “OK” is pressed, it calls the function startNewGame(), similarly called once you load the program for the first time after the bits of Java Swing is rendered on line 50 of Figure 28. ChatGPT programmed the image to be set to the starting hangman0.png via calling the updateHangmanImage(0) on line 26 outside the method startNewGame(). This in turn makes it so line 26 only happens once and never again stopping the image to be reset.

```
95          if (updatedWord.indexOf("_") == -1) {
96              JOptionPane.showMessageDialog(this, "Congratulations! You guessed the word: " + currentWord);
97              startNewGame();
98          } else if (wrongGuesses == 6) {
99              JOptionPane.showMessageDialog(this, "You have run out of guesses! The word was: " + currentWord);
100             startNewGame();
101         }
102     }
```

Figure 29

Finally, the last noteworthy part of the code that not only ChatGPT utilized, but also all of the other AIs, was the use of JOptionPanes to display if you won or not and the random word. As shown in Figure 29, if each of the underscores have been replaced it displays a JOptionPane to say congrats, otherwise if the wrongGuesses is equal to 6, it displays a message saying you lose. Looking back at my code, to keep things simple I displayed a similar message in the console. I will have to give it to AI on this one simply due to the user interface and giving the player the option to play again.

Anticipating Tomorrow: Reflections and Insights

The future of generative AI promises to redefine the landscape of software development and beyond, offering solutions that range from automating mundane tasks to pushing the boundaries of creativity. As AI models continue to evolve and improve, they are increasingly capable of understanding complex structures and generating high-quality output across various domains. In programming, this means that generative AI could potentially streamline the development process, assist developers in writing more efficient code, and even generate novel solutions to intricate problems. In the context of this experiment, the success of ChatGPT in accurately replicating a Java program signifies a significant milestone in AI-driven software development. It demonstrates the capability of generative models to comprehend and synthesize complex code structures effectively. This success is not merely about mimicking syntax but understanding the underlying logic and requirements of the task at hand efficiently. ChatGPT's ability to produce a functional Java program suggests a level of comprehension and adaptability that bodes well for the future of AI in programming.

Conversely, the failures encountered with other AI models such as Gemini, Copilot, and BlackBox offer valuable insights into the challenges that still need to be addressed. These failures could stem from various factors, including limitations in language understanding, inadequate training data, or inherent biases within the models themselves. By dissecting these shortcomings, researchers can pinpoint specific areas for improvement and devise strategies to overcome them. This iterative process of refinement is essential for pushing the boundaries of generative AI and enhancing its reliability and accuracy in real-world applications. Overall, this experiment serves as a crucial steppingstone in advancing the capabilities of generative AI in programming tasks. By providing empirical evidence of performance and highlighting areas for improvement, it guides future research efforts towards addressing key challenges and pushing the boundaries of what AI can achieve in software development. As generative AI continues to evolve, experiments like this play a vital role in shaping its trajectory and unlocking its full potential.

Reflecting on the experiment, it becomes evident that the advancements being made in AI allowing for complex and comprehensive code hold transformative potential for not just the development process. In healthcare, generative AI can revolutionize diagnostic processes by analyzing vast amounts of medical data to identify patterns and anomalies, aiding physicians in making accurate diagnoses and suggesting personalized treatment plans. Additionally, AI-driven tools can streamline administrative tasks, allowing healthcare professionals to focus more on patient care. In education, generative AI can personalize learning experiences by adapting curriculum and teaching methods to individual student needs, fostering better engagement and comprehension. Moreover, AI tutors can provide instant feedback and support, enhancing student learning outcomes. In the creative industries, generative AI can inspire innovation by assisting artists, writers, and designers in generating novel ideas, designs, and compositions. By automating routine tasks, AI frees up creative professionals to focus on higher-level conceptualization and experimentation. Overall, the transformative potential of generative AI lies in its ability to augment human capabilities, improve efficiency, and unlock new possibilities across various domains, ultimately leading to profound advancements in different sections of life.

Generative AI holds immense promise in revolutionizing various sectors, yet it also raises significant concerns regarding misuse, ethical considerations, and the potential impact on jobs and privacy. The ability of generative AI to autonomously create highly complex and comprehensive code introduces the risk of malicious actors exploiting such technology for nefarious purposes, such as developing sophisticated malware or conducting cyberattacks. Ethical considerations arise regarding the responsible use of AI, including ensuring transparency, fairness, and accountability in its deployment. Moreover, the automation of tasks traditionally performed by humans raises apprehensions about job displacement and the need for reskilling and upskilling workers to adapt to evolving roles in an AI-driven economy. Additionally, the vast amounts of data required to train generative AI models pose significant privacy risks, as sensitive information may be inadvertently exposed or exploited without adequate safeguards in place. Addressing these challenges requires a multidisciplinary approach involving policymakers,

technologists, ethicists, and society at large to develop robust regulatory frameworks, ethical guidelines, and responsible AI practices to mitigate potential harms and maximize the benefits of generative AI while safeguarding individual rights and societal well-being.

Through my experiment of tasking four different generative AI to create the same program in Java, I learned several valuable lessons. Firstly, I gained a comprehensive understanding of the capabilities and limitations inherent in various generative AI models. Witnessing how different AI systems approached the identical task allowed me to discern their unique strengths and weaknesses, which is crucial knowledge for leveraging AI in future projects. This insight may guide me in selecting the most appropriate AI tool for specific programming tasks based on their performance and suitability. Secondly, comparing the outputs of each AI-generated program provided me with an in-depth analysis of common errors and challenges encountered during automated code generation. By identifying recurring issues across different AI-generated solutions, I gained valuable insights into potential pitfalls and areas for improvement in automated software development processes. This understanding can inform strategies for refining AI algorithms or incorporating additional error-handling mechanisms to enhance the reliability and robustness of code generated by AI systems.

Furthermore, the process of manually creating the program before tasking the AI deepened my understanding of the underlying algorithms and logic required for the task at hand. Engaging in manual coding prior to AI involvement honed my algorithmic thinking and problem-solving skills, which are essential attributes for proficient software development. Comparing my manual solution to the AI-generated ones enabled me to discern disparities in approach, efficiency, and effectiveness, thereby facilitating a nuanced assessment of the AI-generated code's quality. Lastly, evaluating the performance of each AI-generated program provided me with valuable experience in assessing code quality using criteria such as readability, efficiency, and correctness. By developing a systematic approach to evaluating AI-generated code, I established a framework for discerning between superior and subpar solutions, enabling more informed decision-making in future software development endeavors. This

experience equips me with the ability to critically evaluate AI tools and algorithms, ensuring their alignment with project requirements and objectives.

Conclusion

Throughout this experiment, I've embarked on a journey that shed light on the capabilities and limitations of generative AI models in the domain of software development. As artificial intelligence continues its rapid evolution, it's imperative to grasp its potential, recognize its current boundaries, and chart a course for its integration into diverse domains. As stated previously, my experiences as a student in regard to utilizing generative AI has grown alongside it’s widespread adoption. The class that had the most well thought description of the rules for Generative AI in assisting students was my CIS 430 – Mobile Platforms for Business under Professor Mazzola. In this class there were specifically allowed and disallowed times to utilize AI which were told per instruction of the assignment, quiz, or exam. In addition, the syllabus had detailed descriptions of the role that generative AI would take throughout the semester. To quote the syllabus, in section 25 entitled AI Policy under subsection 25.1.2, it stated,

> “If you use AI, you must provide a citation. There are no formal standards yet for citing ChatGPT or such tools, so we will use the following format:
>
> Source: Tool, version, data, prompt

On top of this, subsection 25.1.3 warns students that if the AI tool used “gets the wrong answer, hallucinates, or steers you in the wrong direction, you bear the responsibility.” I found the Professor Mazzola’s desire for students to be aware of the case-by-case opportunities that AI could assist to be a unique stance by a professor in the academic world. He spent the time to showcase the perks of the paid version of ChatGPT and made it clear that generative AI was not going anywhere and that we have to adopt it or be left behind.

Although this experiment highlighted the success behind generative AI in regard to coding, there is some limitations that require attention. Specifically, in the context of students utilizing generative AI, it is crucial to avoid excessive reliance, which may undermine the essence of this study. My objective was to showcase how AI can assist people in regard to coding by demonstrating its capability to comprehend complex code. Although I tasked the AI with generating code, the purpose was to evaluate its ability to produce intricate, functional code, rather than to complete assignments. Therefore, if you are a student reading this, it's essential to review your course guidelines regarding the use of generative AI and utilize it responsibly to enhance your learning experience and academic performance. Generative AI should be viewed as a tool to augment learning, not as a replacement for academic effort. It is here to help you… not become you.

The success I witnessed with ChatGPT in accurately replicating a Java program marks a significant milestone in AI-driven software development. Its ability to grasp intricate code structures and deliver functional solutions is a testament to the progress achieved in generative AI. However, my experiment also brought to light the hurdles faced by other AI models like Gemini, Copilot, and BlackBox, emphasizing the need for ongoing research and refinement. The setbacks encountered with these models offer valuable insights into areas ripe for improvement, whether it's enhancing language understanding, refining training data quality, or mitigating inherent biases. Tackling these challenges demands a collaborative effort involving researchers, developers, and stakeholders to advance the capabilities of generative AI and ensure its reliability and accuracy in practical applications. Moreover, this experiment afforded me a deeper understanding of the intricacies involved in automated code generation. By comparing outputs and dissecting common errors, I gleaned insights into potential pitfalls and avenues for

enhancement in the automated software development process. This knowledge informs future research endeavors and guides strategies for refining AI algorithms to align with project requirements and objectives effectively.

Furthermore, the experiment highlighted the indispensable role of human involvement in software development, even amidst the rise of AI dominance. While generative AI shows promise in streamlining certain tasks, human expertise remains essential in guiding the development process, detecting errors, and upholding code quality. The synergy between human intelligence and AI capabilities holds the key to unlocking the full potential of software development in the digital era. In conclusion, while generative AI has made significant strides in emulating human-like abilities in software development, there's still substantial ground to cover. By acknowledging both the triumphs and the challenges encountered in this experiment, I lay the groundwork for a more nuanced understanding of AI's impact on the future of software development. Through sustained research, collaboration, and responsible implementation, we can harness the transformative potential of generative AI to forge a more efficient, innovative, and equitable digital landscape for generations to come.

Sources

Office, U.S. Government Accountability. “Artificial Intelligence’s Use and Rapid Growth Highlight Its Possibilities and Perils.” *U.S. GAO*, 10 Nov. 2022, www.gao.gov/blog/artificial-intelligences-use-and-rapid-growth-highlight-its-possibilities-and-perils.

Office, U.S. Government Accountability. “Science & Tech Spotlight: Generative AI.” *Science & Tech Spotlight: Generative AI | U.S. GAO*, 6 Sept. 2023, www.gao.gov/products/gao-23-106782.

Arizona State University. (2023). *CIS 430: Mobile Platforms for Business course syllabus.* Tempe, Arizona: Daniel Mazzola.